



UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA

Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Laboratorio No. 6 · Múltiples clientes y API Keys como hash · API_TEATRO

ESPACIO ACADÉMICO:

Seguridad en el Desarrollo de Software

PRESENTADO A:

Sergio Arley Puerto Moreno

AUTORES:

Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. El problema

Hasta el laboratorio anterior la API tenía una sola clave global en el `.env`. Funcionaba, pero tenía dos límites.

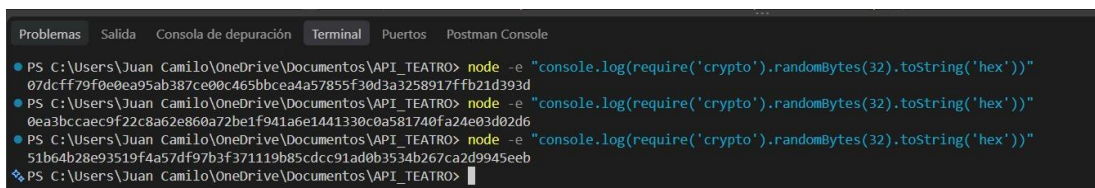
El primero es que no sabía quién hacía cada petición. Todos los clientes usaban la misma clave, así que el servidor no podía distinguir si la petición venía de Postman, de la taquilla o de una aplicación móvil.

El segundo es que no podía quitarle el acceso a uno sin quitárselo a todos. Cambiar la clave dejaba fuera a todo el mundo.

En este laboratorio pasamos a tres clientes, cada uno con su clave, y dejamos de guardar las claves en claro.

2. Los tres clientes

Generamos tres claves ejecutando tres veces el mismo comando de Node, que trae un generador criptográficamente seguro.

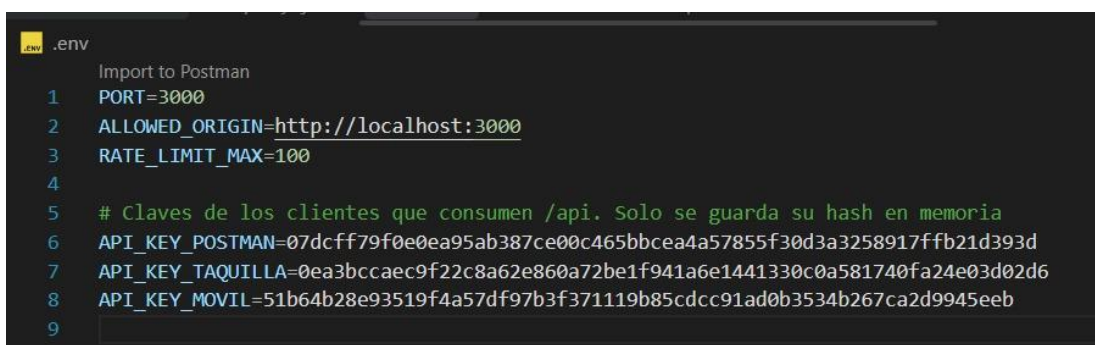


```
PS C:\Users\Juan Camilo\OneDrive\Documentos\API_TEATRO> node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
07dcff79f0e0ea95ab387ce00c465bbcea4a57855f30d3a3258917ffb21d393d
PS C:\Users\Juan Camilo\OneDrive\Documentos\API_TEATRO> node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
0ea3bccac9f22c8a62e860a72be1f941a6e1441330c0a581740fa24e03d02d6
PS C:\Users\Juan Camilo\OneDrive\Documentos\API_TEATRO> node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
51b64b28e93519f4a57df97b3f371119b85cdcc91ad0b3534b267ca2d9945eeb
PS C:\Users\Juan Camilo\OneDrive\Documentos\API_TEATRO>
```

Figura 1. Las tres claves generadas. Cada una son 32 bytes en hexadecimal, o sea 64 caracteres.

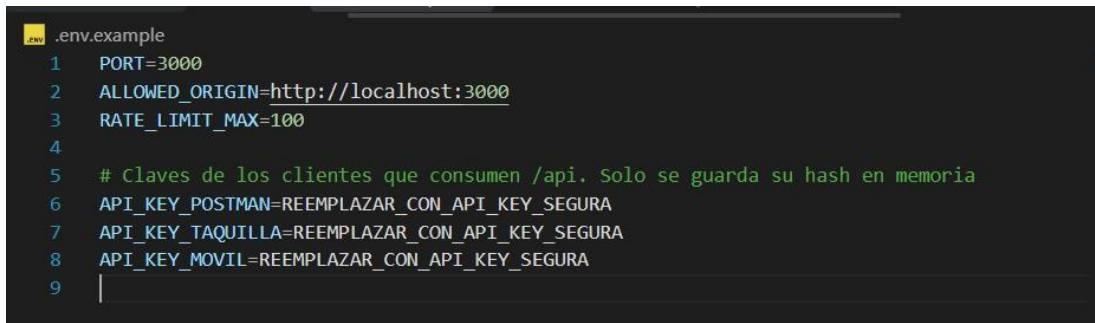
Quedaron registrados tres clientes. El primero es Postman Laboratorio, con la clave `API_KEY_POSTMAN`, activo. El segundo es Taquilla del Teatro, con `API_KEY_TAQUILLA`, también activo. Y el tercero es Aplicación Móvil, con `API_KEY_MOVIL`, que nace deshabilitado a propósito, para poder probar el caso de un cliente revocado.

El laboratorio de referencia usa una aplicación administrativa como segundo cliente. En el teatro lo cambiamos por la taquilla, que es quien vende boletas en la puerta.



```
.env
Import to Postman
1 PORT=3000
2 ALLOWED_ORIGIN=http://localhost:3000
3 RATE_LIMIT_MAX=100
4
5 # Claves de los clientes que consumen /api. Solo se guarda su hash en memoria
6 API_KEY_POSTMAN=07dcff79f0e0ea95ab387ce00c465bbcea4a57855f30d3a3258917ffb21d393d
7 API_KEY_TAQUILLA=0ea3bccac9f22c8a62e860a72be1f941a6e1441330c0a581740fa24e03d02d6
8 API_KEY_MOVIL=51b64b28e93519f4a57df97b3f371119b85cdcc91ad0b3534b267ca2d9945eeb
9
```

Figura 2. El `.env` con las tres claves. La `API_KEY` vieja ya no está.



```

.env.example
1  PORT=3000
2  ALLOWED_ORIGIN=http://localhost:3000
3  RATE_LIMIT_MAX=100
4
5  # Claves de los clientes que consumen /api. Solo se guarda su hash en memoria
6  API_KEY_POSTMAN=REEMPLAZAR_CON_API_KEY_SEGURA
7  API_KEY_TAQUILLA=REEMPLAZAR_CON_API_KEY_SEGURA
8  API_KEY_MOVIL=REEMPLAZAR_CON_API_KEY_SEGURA
9

```

Figura 3. El `.env.example`, con las mismas tres variables y un valor de relleno. Este archivo sí se sube al repositorio.

3. Por qué guardo el hash y no la clave

Al arrancar, el servidor calcula el hash SHA-256 de cada clave y solo se queda con el hash. La clave original no se guarda en ningún sitio.

Un hash no se puede deshacer. Si alguien lee `src/data/apiKeys.js` o mira los datos en memoria, solo ve tres hashes, y un hash no sirve como clave. Lo comprobamos pasándole al service el hash guardado de Postman como si fuera la clave: devolvió `null`, que el middleware convierte en 401. El service calcula el hash de lo que recibe, y el hash de un hash no coincide con nada.

Para comprobar una petición tampoco hace falta la clave original: calculamos el hash de lo que llega y lo comparamos con los guardados.

Una aclaración sobre el método. SHA-256 sin sal funciona aquí porque las claves son 32 bytes aleatorios, imposibles de adivinar probando. Con contraseñas escritas por personas no bastaría, porque la gente repite las mismas y se pueden romper con diccionarios. Eso es lo que se ve en el siguiente laboratorio.

4. Las capas nuevas

`src/utils/crypto.util.js` tiene `generarHash`, que calcula el SHA-256 de un texto, y `compararSeguro`, que compara dos valores en tiempo constante. Toda la parte criptográfica quedó aquí.

`src/data/apiKeys.js` simula la tabla que después será PostgreSQL. Primero comprueba que existan las tres variables del `.env` y después arma los tres registros con id, cliente, hash, activa y creadaEn.

`src/services/apiKeys.service.js` tiene `buscarClientePorApiKey`: calcula el hash de lo recibido, lo compara con cada registro y devuelve el que coincida, o `null`.

El middleware lo reescribimos entero. Ya no usa `crypto` ni tiene su propia copia de `compararSeguro`: solo le pregunta al service quién es el cliente y decide qué responder. En todo `src/` quedó una sola función de comparación.

El orden de dependencias es este:

`app.js` → `middleware` → `service` → `data` → `crypto.util`

5. Los cuatro estados

Hasta ahora la API solo respondía 401 cuando algo iba mal con la clave. Ahora hay cuatro situaciones distintas y dos códigos.

- No llega la cabecera: **401**, "API Key requerida".
- La clave no es de ningún cliente: **401**, "API Key inválida".
- La clave es de un cliente deshabilitado: **403**, "API Key deshabilitada".
- La clave es de un cliente activo: la petición sigue su curso.

La diferencia entre 401 y 403 importa. El 401 significa "no sé quién eres": no llegó clave, o llegó una que no reconozco. El 403 significa "sé quién eres, pero no te dejo pasar": la clave es correcta y el servidor sabe que es la Aplicación Móvil, pero ese cliente está deshabilitado.

El 403 es justo lo que permite revocar un cliente sin tocar a los demás. Postman y la taquilla siguen entrando.

Probamos los cuatro contra `GET /api/asistentes`.

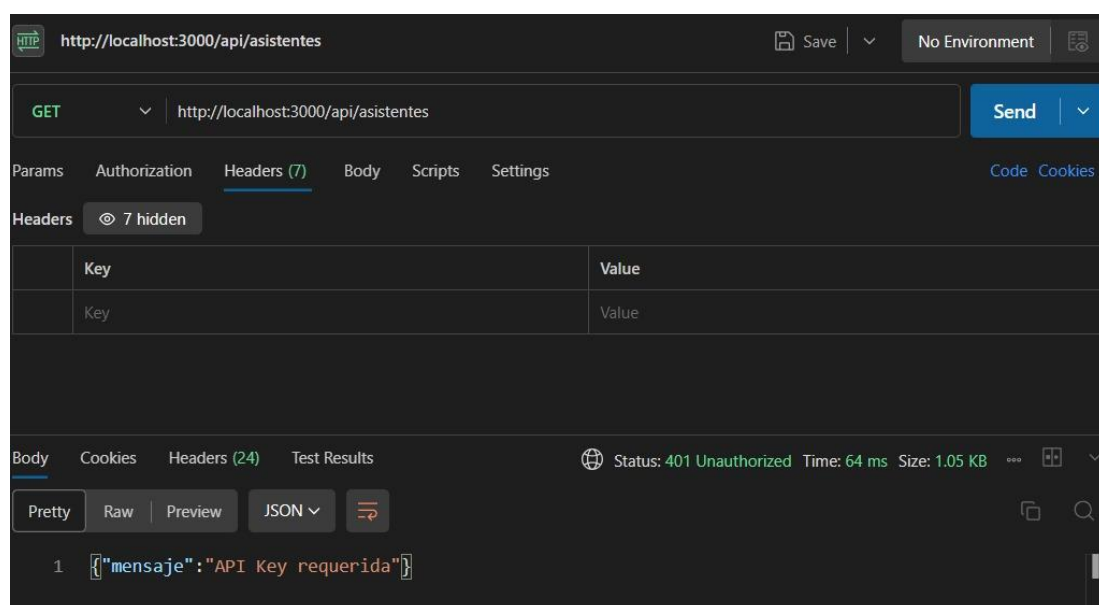


Figura 4. Sin la cabecera X-API-Key. Responde 401, "API Key requerida".

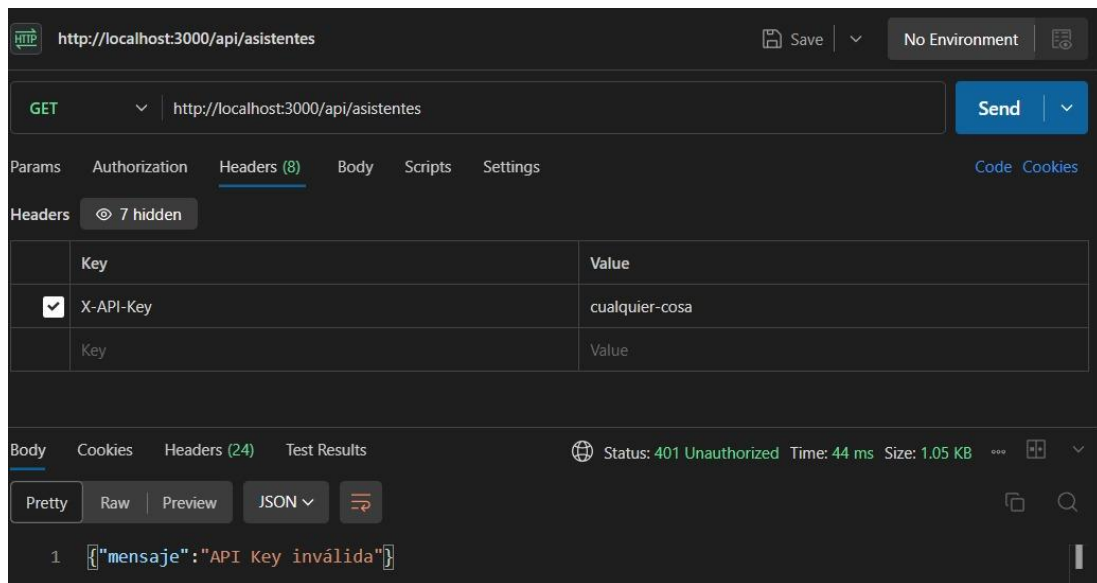


Figura 5. Con una clave inventada. Responde 401, pero el mensaje cambia: "API Key inválida".

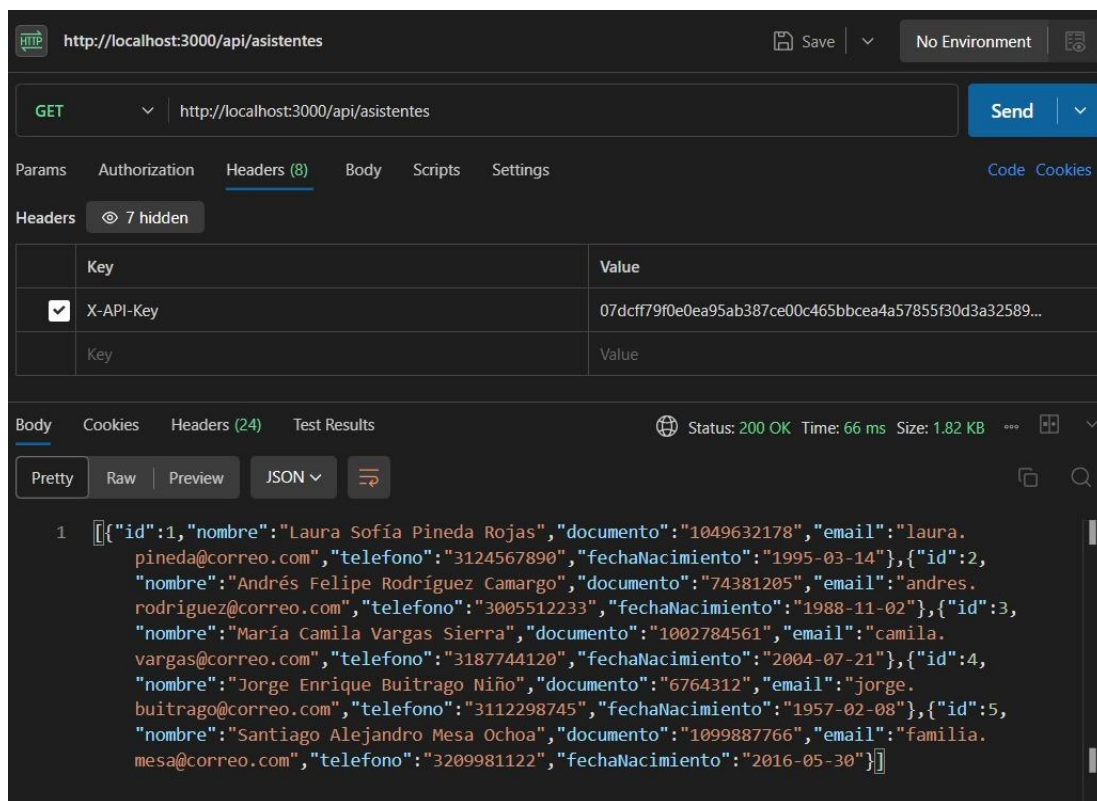


Figura 6. Con la clave de Postman Laboratorio. Responde 200 con los cinco asistentes.

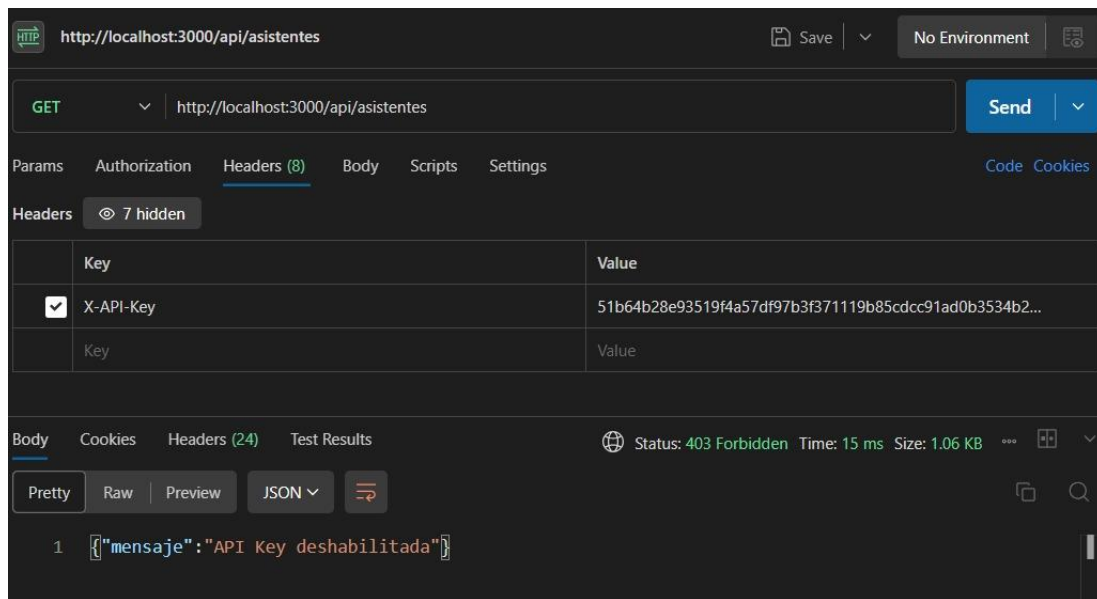


Figura 7. Con la clave de Aplicación Móvil. Responde 403: la clave es correcta, pero el cliente está deshabilitado.

También comprobamos que la clave no reemplaza lo anterior: con una clave válida, borrar una función con boletas sigue dando 409.

6. La configuración ahora se revisa al arrancar

Antes, si faltaba la variable `API_KEY`, el servidor arrancaba igual y el fallo aparecía con la primera petición, que devolvía 500. Lo vimos pasar en este mismo laboratorio: al quitar la variable vieja y antes de reescribir el middleware, todo `/api` respondía 500, con clave o sin ella.

Ahora la comprobación está en `src/data/apiKeys.js` y se ejecuta al cargar el archivo. Si falta alguna de las tres variables, lanza un error y el servidor no llega a arrancar. Lo comprobamos dejando `API_KEY_MOVIL` vacía: el proceso terminó con código 1 y el mensaje **Faltan variables de entorno para las API Keys**, sin ponerse nunca a escuchar.

Es mejor así. Un servidor mal configurado no debería arrancar y esperar a que la primera petición descubra el problema.

Esto depende del orden de carga. `apiKeys.js` lee `process.env` en cuanto se importa, así que `dotenv` tiene que haber corrido antes. En `app.js` esa es la primera línea, antes de cualquier otro `require`, así que no hubo problema. Lo comprobamos también al revés: cargando el archivo sin `dotenv`, lanza el error aunque las variables estén bien puestas.

7. El endpoint que dice quién eres

Cuando la clave es válida, el middleware deja en la petición quién es el cliente:

```
req.clienteApi = { id: cliente.id, nombre: cliente.cliente };
```

Creamos `src/routes/seguridad.routes.js` con una sola ruta, `GET /cliente`, que devuelve eso. No necesita su propio middleware: como `validarApiKey` está montado sobre todo `/api`, la ruta queda protegida sola.

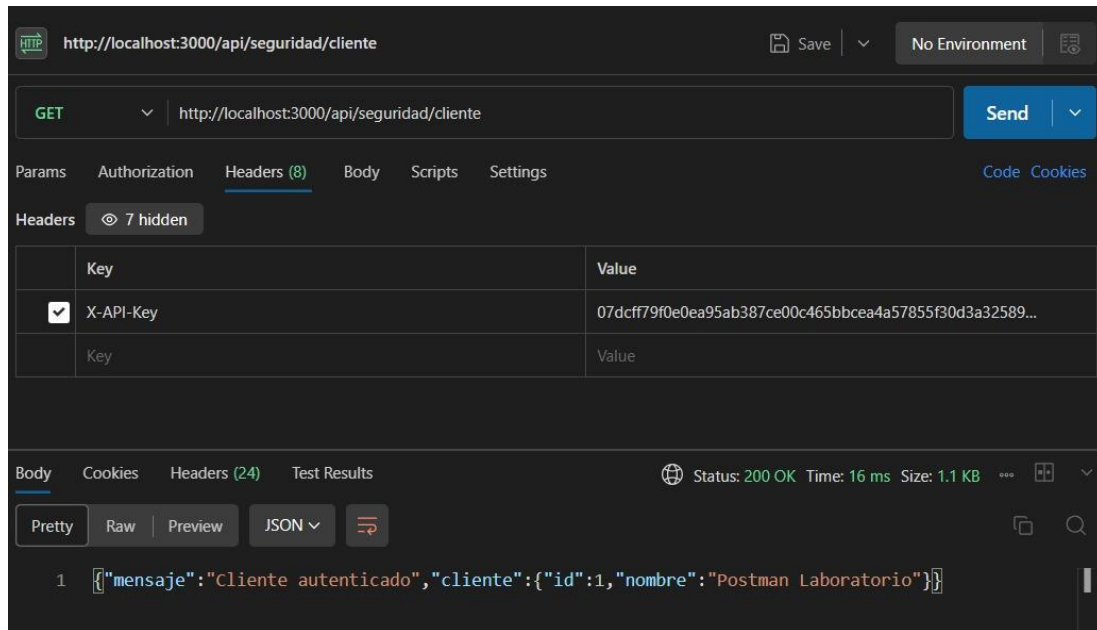


Figura 8. `GET /api/seguridad/cliente` con la clave de Postman Laboratorio. Responde con el cliente 1.

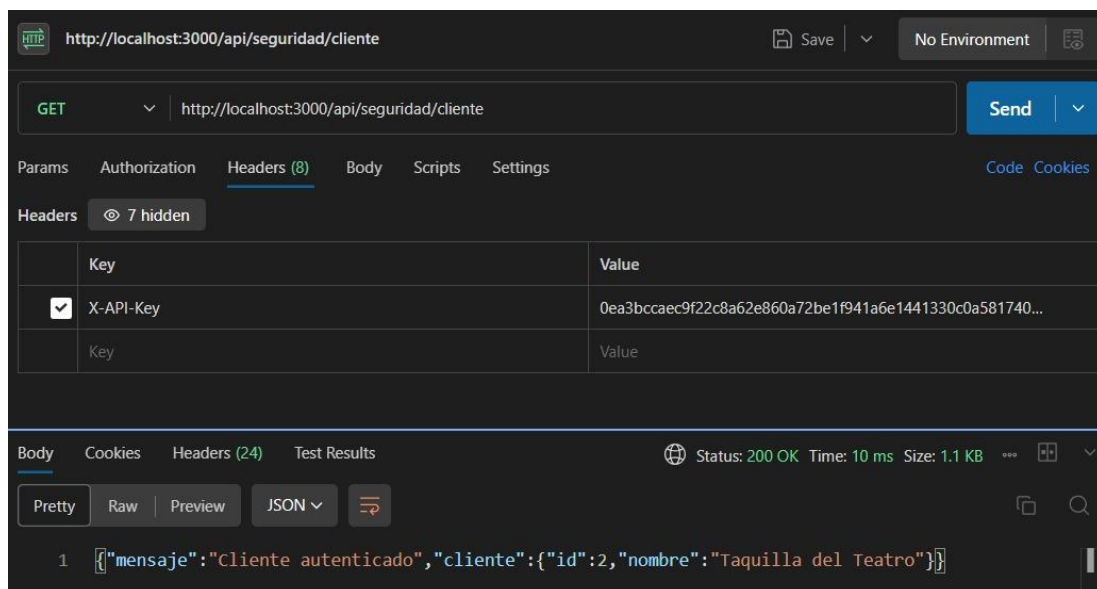


Figura 9. La misma URL, con la clave de Taquilla del Teatro. Responde con el cliente 2.

Esas dos figuras son lo que este laboratorio quiere demostrar. La URL es exactamente la misma, el método también. Lo único que cambió fue la cabecera, y el servidor supo decir quién estaba preguntando. Con una sola clave global eso era imposible.

La misma URL con la clave de Aplicación Móvil responde 403, y sin cabecera responde 401.

8. Lo que casi no cambió

En `app.js` solo añadimos dos líneas: la importación del router de seguridad y su montaje. El resto es igual que en el laboratorio anterior. `app.js` ni siquiera sabe que ahora hay tres claves, ni que se guardan como hash.

En Swagger no tocamos la definición de seguridad. Sigue diciendo lo mismo: el cliente manda una API Key en la cabecera `X-API-Key`. Da igual si detrás hay una clave o trescientas. Lo único que agregamos fue el tag para la ruta nueva.

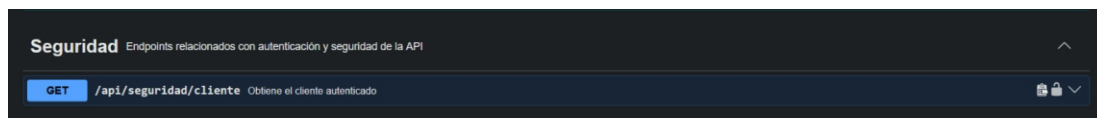


Figura 10. El grupo Seguridad en Swagger, con la ruta nueva dentro.

Que el cambio quedara contenido en cuatro archivos, sin tocar rutas, controladores ni servicios de negocio, es consecuencia de que cada capa hace una sola cosa.

Después del cambio el spec tiene 39 endpoints en 19 rutas: los 38 que había más el nuevo. Siguen los 30 schemas.

9. Verificación

Repetimos las dos herramientas al terminar. Semgrep corrió 73 reglas sobre 35 archivos con **0 hallazgos**, y npm audit tampoco encontró vulnerabilidades. No añadimos dependencias, porque `crypto` viene con Node.

Un detalle del escaneo: los cuatro archivos nuevos todavía no tenían commit y entraron igual, porque Semgrep excluye lo que está en `.gitignore`, no lo que falta por subir. El conteo lo confirma, porque pasó de 31 a 35 archivos.

Las pruebas automatizadas

Los siete scripts de `pruebas/` leían la clave de `process.env.API_KEY`. Al quitar esa variable quedaron sin clave, y además en ese momento todo `/api` respondía 500 por el middleware viejo.

No llegamos a ejecutarlos en ese estado, así que no tenemos un número de pruebas rotas. Los arreglamos antes de correrlos, cambiando en los siete la variable por `API_KEY_POSTMAN`. Ninguno tiene una clave escrita: todos la leen del `.env`. Después del cambio volvieron a pasar las 224.

10. Lo que todavía no sé

Con esto sabemos qué aplicación consume la API: Postman, la taquilla o la aplicación móvil. No sabemos qué persona la está usando. Si alguien vende una boleta desde la taquilla, la API sabe que fue la taquilla, no quién estaba en el mostrador.

Tampoco hay roles. Un cliente activo puede hacer todo lo que la API permite.

Para saber quién es la persona hacen falta usuarios con contraseña, y después tokens y roles. Eso queda para el siguiente laboratorio.

11. Repositorio

<https://github.com/JuanK2550/API-TEATRO>

El trabajo está en el commit [57243ec](#), y las salidas de Semgrep y npm audit en [9b5cc5b](#).

Antes de subir comprobamos que el `.env` no estuviera en el repositorio ni en el historial, y que ninguna de las tres claves apareciera en ningún archivo ni en ningún cambio.

Las claves que se ven en las figuras 1 y 2 ya no están en uso. Generamos tres nuevas después de documentar.